

IFT222/CSC227 COMPUTER ARCHITECTURE & ORGANIZATION
TEST2 MARKING GUIDE

1. Assume a computer has 32-bit addresses. Each block stores 16 words. A direct-mapped cache has 256 blocks. In which block (line) of the cache would we look for each of the following addresses? Addresses are given in hexadecimal for convenience. 6mks

- a. 1A2BC012
- b. FFFF00FF
- c. 12345678
- d. C109D532

Solution

Of the 32 bit address, the last four bits denote the word on the line. Since four bits is used for one hex digit, the last digit of the address is the word on the line. With 256 blocks in the cache, we need 8 bits to denote the block number. This would be the third to last and second to last hex digit.

Step 1: Determine number of bits required by each of Offset, Block/Line Number and Tag Number

- 16 words = 2^4 words = $\log_2 2^4 = 4 \log_2 2 = 4$ bits for Offset
- Block/Line Number: the given 256 blocks would **require 8 bits** i.e $256 = 2^8$
- For Tag Number = $32 - (8+4) = 20$ bits

Step 2: Convert all addresses to binary

- 1A2BC012 = 0001 1010 0010 1011 1100 0000 0001 0010
- FFFF00FF = 1111 1111 1111 1111 0000 0000 1111 1111
- 12345678 = 0001 0010 0011 0100 0101 0110 0111 1000
- C109D532 = 1100 0001 0000 10001 1101 0101 0011 0010

Tag Number (bit)	Block/line number (bit)	Offset/Word Size (bit)
20	8	4

- a. this would be block 01 (0000 0001) which is block 1
- b. this would be 0F (0000 1111) which is block 15
- c. this would be 67 (0110 0111) which is block 103 (remember, 67 is a hex value)
- d. this would be 53 (0101 0011) which is block 83.

2. A computer has an 8 GByte memory with 64-bit word sizes. Each block of memory stores 16 words. The computer has a direct-mapped cache of 128 blocks. The computer uses word level addressing. What is the address format? If we change the cache to a 4-way set associative cache, what is the new address format? (draw the layout for each format) 3mks

Given:

- **Main Memory:** 8 GB = 2^{32} Bytes
- **Word size:** 64 bits = 8 Bytes $\Rightarrow$ Each **word = 8 bytes**
- $\Rightarrow$ Total number of words = $2^{32} / 2^3 = 2^{30}$ words
- **Addressing:** Word-level addressing $\Rightarrow$ addresses range from 0 to $2^{30}-1$
- So, **address size = 30 bits**

- **Block size:** 16 words = $16 \times 8 = 128$ bytes
- **Direct-Mapped Cache:**
 - Number of cache blocks = 128
 - Cache block index = $\log_2(128) = 7$ bits

1. Address Format for Direct-Mapped Cache

Breakdown:

- **Word Address Size:** 30 bits
- **Block size = 16 words** $\Rightarrow$ **Offset within block** = $\log_2(16) = 4$ bits
- **Number of blocks in cache = 128** $\Rightarrow$ **Index bits** = $\log_2(128) = 7$ bits
- **Tag bits = Remaining** = $30 - 7 - 4 = 19$ bits

Address format:

Tag (19 bits)	Index (7 bits)	Block Offset (4 bits)
---------------	----------------	-----------------------

2. Address Format for 4-Way Set Associative Cache

Breakdown:

- Cache has 128 blocks and is 4-way set associative $\Rightarrow$ Number of sets = $128 / 4 = 32$ sets
- Set index = $\log_2(32) = 5$ bits
- Block offset remains the same = 4 bits (for 16 words)
- Tag = $30 - 5 - 4 = 21$ bits

New address format:

Tag (21 bits)	Set Index (5 bits)	Block Offset (4 bits)
---------------	--------------------	-----------------------

3. What do you understand by the term 'Addressing Mode'? 1mk

An addressing mode is a method used in computer architecture to specify how the operand (data) for an instruction is obtained or accessed.

In simpler terms, it defines how the CPU should interpret the address or data specified in an instruction—whether it's a direct value, a memory location, or derived using a register.

4. Describe the following types of Addressing Mode. Use diagrams where necessary 6mks

- Immediate Addressing mode
- Direct Addressing Mode
- Register Addressing Mode

Immediate Addressing Mode

Description:

In **Immediate Addressing Mode**, the operand (data) is **explicitly specified** in the instruction itself. That means the value to be used is **part of the instruction**, not fetched from memory or a register.

Example:

MOV R1, #5

This means: **Load the value 5 directly into register R1.**

Diagram:

Instruction: MOV R1, #5

```
+-----+-----+
| Opcode (MOV) | Operand (#5) |
+-----+-----+
```

Result:

$R1 \leftarrow 5$

Key Characteristics:

- Fast (no memory access needed)
- Operand is a constant
- Common in setting initial values

2. Direct Addressing Mode (Absolute Addressing)

Description:

In Direct Addressing Mode, the instruction contains the memory address of the operand. The CPU fetches the data directly from that memory location.

Example:

assembly

LOAD R1, 3000

This means: **Load the contents of memory location 3000 into register R1.**

Diagram:

Instruction: LOAD R1, 3000

```
+-----+-----+
| Opcode (LOAD) | Address (3000) |
+-----+-----+
```

Memory[3000] → R1

Key Characteristics:

- One memory access is needed to get the operand.
- Simple and direct.
- Useful when data location is fixed.

3. Register Addressing Mode

Description:

In Register Addressing Mode, the operand is stored in a register, and the instruction specifies the register name.

Example:

ADD R1, R2

This means: **Add the contents of R2 to R1.**

Diagram:

Instruction: ADD R1, R2

```

+-----+-----+
| Opcode (ADD) | Operands (R1, R2)|
+-----+-----+

```

$R1 \leftarrow R1 + R2$

Key Characteristics:

- Fastest (no memory access required)
- Operands are already in CPU registers
- Efficient for arithmetic/logical operations

5. a. Describe any two categories of Instructions that we have. Give two examples for each. 4mks
 b. Use **One and Two address instruction formats** to write a Pentium 4 assembly language program segment for the expression below.

$$X = ((A * B) + C/D) / (F * G + J * K)$$

5mks

One address	Two address
LOAD F	MOV R1, A
MUL G	MUL R1, B
STORE T	MOV R2, C
LOAD J	DIV R2, D
MUL K	ADD R1, R2
ADD T	MOV R3, F
STORE T	MUL R3, G
LOAD A	MOV R2, J
MUL B	MUL R2, K
STORE R	ADD R2, R3
LOAD C	DIV R1, R2
DIV D	MOV X, R1
ADD R	
DIV T	
STORE X	
2½mks	2½mks